

VersualRL: Closed-Loop Verbal Reinforcement Learning with Visual Execution Feedback for Task-Level Robot Planning

Dmitrii Plotnikov*, Iaroslav Kolomiets*, Dmitrii Maliukov*, Dmitrii Kosenkov*,
 Daniia Zinniatullina, Artem Trandofilov, Georgii Gazaryan, Kirill Bogatikov, Timofei Kozlov,
 Mikhail Konenkov, Miguel Altamirano Cabrera, and Dzmitry Tsetserukou

Abstract— We introduce VersualRL, a closed-loop framework for task-level robot planning that uses visual execution feedback to iteratively refine executable Behavior Trees through structured natural-language critique. VersualRL combines a vision-language model critic, which analyzes visual observations and Behavior Tree execution traces, with a large language model actor that performs discrete and interpretable policy updates. During physical deployment, both foundation models remain frozen, while the executed Behavior Tree is updated at the symbolic level without online gradient-based policy optimization. This enables transparent reasoning, explicit causal feedback, and human-interpretable policy evolution. We validate VersualRL on a real mobile robot performing a multi-stage navigation and manipulation task under execution uncertainty. Experimental results show that the framework supports explainable policy improvements, closed-loop adaptation to execution failures, and successful deployment on physical robotic systems.

I. INTRODUCTION

In real-world environments, mobile robots operate under significant execution uncertainty [1]. Variations in layouts, imperfect localization, and low-level control errors make the design of robust task-level policies challenging and time-consuming [2]. In practice, such policies are often implemented using Behavior Trees (BTs), which provide modularity and transparency but require significant manual tuning to handle unexpected failures [3], [4].

Reinforcement Learning (RL) offers a principled framework for autonomous policy improvement through interaction and iteration. Recent Deep RL (DRL) and Multi-Agent Deep RL (MADRL) methods have demonstrated strong performance in robotic motion planning and task allocation [5], [6]. However, most RL approaches optimize policies in a sub-symbolic space, producing behaviors that are difficult to interpret or debug [7]. This lack of transparency limits their applicability for the real-world deployment of robotic systems.

Recent works [8], [9], [10], [11], [12] have explored the use of Large Language Models (LLMs) for robotic task planning, leveraging their reasoning and generalization capabilities [13], [14]. While promising, LLM-based planners typically operate in an open-loop manner and lack grounded

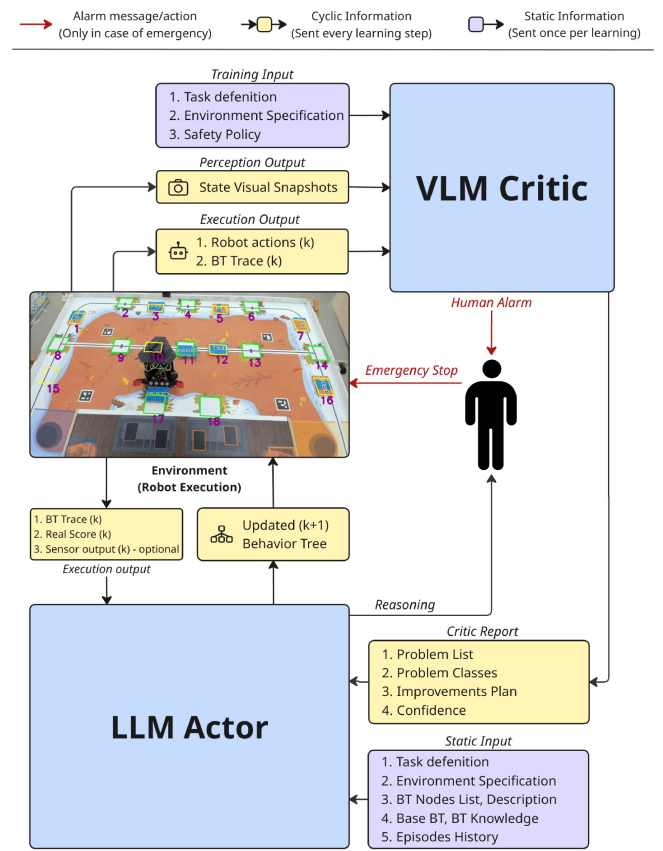


Fig. 1. Closed-loop VersualRL architecture. The VLM critic analyzes visual snapshots and BT execution traces to produce structured feedback, while the LLM actor updates the executable BT using critic feedback and the human-verified episode score R .

feedback from the physical world. As a result, they struggle to reliably adapt to real-world execution failures.

To address these limitations, we propose VersualRL, a closed-loop framework for task-level robot planning that combines verbal policy refinement with visual execution feedback. In VersualRL, task policies are represented as executable Behavior Trees and iteratively refined through structured natural-language feedback. An LLM-based actor modifies the task plan, while a VLM critic analyzes robot operation using visual observations and BT execution traces [15]. Unlike Reflexion-style verbal self-refinement, VersualRL grounds feedback in physical robot execution;

*These authors contributed equally to this work.

The authors are with the Intelligent Space Robotics Laboratory, Skolkovo Institute of Science and Technology. {dmitrii.plotnikov2, iaroslav.kolomiets, dmitrii.maliukov, dmitrii.kosenkov, daniia.zinniatullina, artem.trandofilov, georgii.gazaryan, kirill.bogatikov, timofei.kozlov, mikhail.konenkov, m.altamirano, d.tsetserukou}@skoltech.ru

unlike VLM-based reward learning such as RL-VLM-F, it directly edits an executable symbolic BT policy instead of learning a dense reward model or optimizing a neural policy.

II. RELATED WORK

Recent advances in language and vision-language foundation models have enabled more flexible robotic perception, planning, and control.

A. LLMs for Task-Level Planning and Code Generation

Recent works employ large language models as high-level planners that translate natural-language instructions into executable code or symbolic task plans. Some systems [16], [17], [18], [19], [20], [21] combine LLM-based common-sense reasoning with task-and-motion planning, converting underspecified goals into commands that can be executed by classical planners. These approaches demonstrate strong semantic reasoning, but they typically operate in a one-shot or open-loop manner and lack systematic mechanisms for strategy refinement.

BTs provide an interpretable and modular representation for LLM-driven planning. Previous research showed that LLMs can synthesize structurally valid BTs from natural-language commands [22], [23]. However, existing approaches focus primarily on initial plan generation and do not address post-deployment policy improvement.

B. Vision-Language Models for Reward Learning and Evaluation

Vision-language models have been widely explored for reward learning in robotics. Some methods [24], [25], [26], [27], [28] derive preference-based rewards from visual comparisons, providing automated supervision for reinforcement learning. These approaches rely on gradient-based optimization in sub-symbolic policy spaces, limiting their interpretability. Other works employ VLMs for affordance extraction, scene understanding, and autonomous driving assistance [29], primarily as perception modules rather than explicit execution critics.

C. Critic-Based Policy Refinement

Recent studies investigate VLMs [30], [31] and agentic RL loops [32] as behavioral critics or self-refining mechanisms that identify undesirable actions and support iterative refinement. However, prior methods typically evaluate or rank policies without directly modifying structured symbolic representations such as Behavior Trees.

In contrast, our work couples a VLM-based visual critic with an LLM actor that performs direct, interpretable updates of executable Behavior Trees. This enables closed-loop task-level policy refinement on a physical robot without gradient-based reward learning or simulation.

III. VERSUALRL FRAMEWORK FOR TASK-LEVEL ROBOTIC PLANNING

This section formalizes our framework and describes the actor-critic interaction, symbolic policy representation, and execution-driven refinement mechanism.

A. Policy Representation

Task policies are represented as executable BTs, which provide modularity and interpretability.

The actor modifies BT structure and parameters directly, operating on a library of predefined expert-designed nodes and subtrees. This allows policy updates to remain transparent, verifiable, and grounded in existing expert knowledge. The initial BT is deliberately simple and does not encode the task strategy, serving only as a basis for iterative improvement.

B. LLM Actor: Symbolic Policy Refinement

The actor is an LLM that operates at the symbolic level, refining an existing BT based on structured feedback. To constrain the search space and ensure syntactic validity of generated policies, the actor is prompted with a structured context consisting of: (i) the task definition, (ii) an environment specification, (iii) a library of available BT nodes with their descriptions, and (iv) BT authoring knowledge (coding rules) together with the current (initial) BT. Importantly, the actor does not receive raw sensory streams and does not directly control the robot; instead, it proposes discrete, interpretable edits to the BT structure and parameters.

Optionally, the actor’s input interface can be augmented with additional structured, perception- or sensor-derived state estimates when available. We treat this as an ablation on actor observability and evaluate it in Section V by providing symbolic block color/orientation information to the actor in addition to critic feedback, demonstrating that such structured inputs can substantially improve learning stability and overall performance.

C. VLM Critic: Visual Execution Feedback

The critic is a VLM that observes task execution through visual snapshots and BT execution traces. Instead of estimating scalar rewards, it outputs structured natural-language feedback grounded in observable evidence. To support different stages of execution, the critic operates in three modes: *Initial*, *Intermediate*, and *Final*.

In addition to the textual critique, the critic outputs two scalars: an alarm score $s \in [0, 1]$ indicating issue severity, and a confidence $c \in [0, 1]$ indicating certainty. The alarm score is supervised using an ordinal severity rubric. In *Initial* mode we annotate $s=0.0$ for clean setup, $s \in [0.1, 0.3]$ for minor imperfections, $s \in (0.3, 0.5)$ for borderline cases, $s \in [0.5, 0.7]$ for actionable problems, and $s > 0.7$ for severe safety-critical issues. For *Intermediate/Final* (Human Alarm) annotations we use the same severity semantics but a slightly more tolerant rubric to account for inevitable manipulation noise, while preserving $s \geq 0.5$ as actionable problems and $s > 0.7$ as severe failures.

After each episode, a human evaluator verifies the physical task outcome and computes the human-verified episode score R according to the fixed scoring rules in Table II. The score is based on predefined task conditions and is not predicted by the critic.

VersualRL operates in an episodic closed loop. At each episode, the current BT is executed on the robot and the critic analyzes observations and traces to produce feedback. If $s \geq 0.5$ or $c < 0.3$, a human operator is notified. The actor then updates the BT using the final critic feedback and the human-verified episode score R . The overall procedure is summarized in Algorithm 1.

Algorithm 1: VersualRL Policy Refinement

Input: Initial Behavior Tree BT_0 , Actor $\mathcal{A}$, Critic $\mathcal{C}$

Output: Refined Behavior Tree BT^*

$BT \leftarrow BT_0$;

Initialize actor memory $H \leftarrow \emptyset$;

for $episode = 1$ **to** N **do**

 Reset critic episodic memory $M \leftarrow \emptyset$;

 Capture initial image I_0 ;

$y_0 \leftarrow \mathcal{C}.INITIAL(I_0)$;

if $ALARM(y_0)$ **then**

 Notify human operator;

while *episode not terminated* **do**

 Execute next subtree of BT ;

 Capture image I_t and BT execution trace T_t ;

$y_t \leftarrow \mathcal{C}.INTERMEDIATE(I_t, T_t)$;

$M \leftarrow M \cup \{(I_t, T_t, y_t)\}$;

if $ALARM(y_t)$ **then**

 Pause execution and wait for operator intervention;

 Capture final image I_f and complete BT execution trace T ;

$F \leftarrow \mathcal{C}.FINAL(I_f, T, M)$;

if $ALARM(F)$ **then**

 Notify human operator;

 Compute human-verified episode score R ;

$BT \leftarrow \mathcal{A}(BT, F, R, H)$;

$H \leftarrow H \cup \{F, R\}$;

$BT^* \leftarrow BT$;

return BT^* ;

Here, F denotes the final critic report, R is the human-verified episode score computed according to Table II, and H stores prior reports and scores.

IV. EXPERIMENTAL SETUP

A. Robotic Platform

The experiments were conducted on a custom mobile manipulation robot designed to perform logistics tasks (Fig. 2).

The robot performs autonomous navigation between pre-defined zones and uses a servo-actuated gripper with an auxiliary vacuum channel to pick, rotate, and place blocks. The manipulation system can handle a fixed batch size of up to four blocks per transport cycle.

B. Experimental Environment

We established certain specifications for the experimental field, including the existence of several load and unload

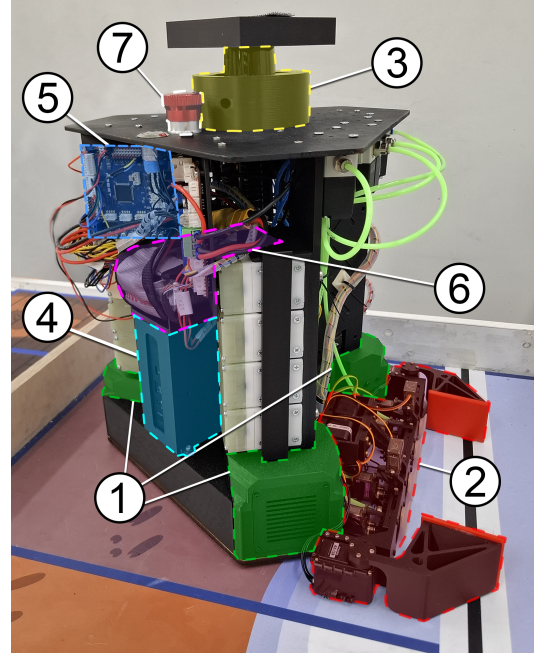


Fig. 2. Hardware architecture of the mobile robot: (1) three-wheeled omnidirectional base, (2) custom servo-actuated gripper with an auxiliary vacuum channel, (3) LiDAR sensor, (4) onboard Intel NUC computer, (5) custom STM32-based low-level controller, (6) LiPo battery, and (7) emergency stop button.

zones, the designated starting area, and a movable object (shelf). For the purpose of experiments, we used the available robotics competition environment (Fig. 3). The geometry of the field provides spatially distributed pickup and delivery zones suitable for warehouse-style manipulation tasks.

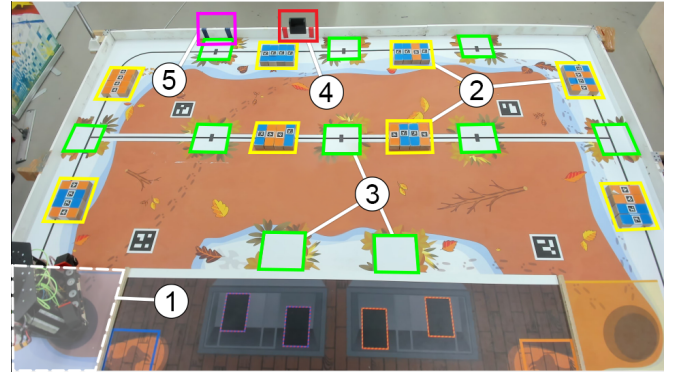


Fig. 3. Layout of the experimental environment: (1) start/finish area, (2) load zones containing block sets, (3) designated unload zones, (4) initial position of the movable shelf, and (5) target position of the movable shelf.

We use wooden blocks of 150 mm $\times$ 50 mm $\times$ 30 mm. Two opposite large faces are painted orange and blue, while the remaining faces are unpainted. The color of the upward-facing side determines the orientation of the block. If the orange face is upward, the block is incorrectly oriented (upside down).

C. Task Definition

The experimental task represents a structured warehouse logistics scenario under a time constraint. The robot is

TABLE I

FINE-TUNING HYPERPARAMETERS FOR THE QWEN2.5-VL-3B CRITIC

Parameter	Value
Base model	Qwen2.5-VL-3B-Instruct (8-bit, bitsandbytes)
LoRA rank r / α	16 / 16
LoRA dropout	0.15
Target modules	Attention and MLP projections
Optimizer	AdamW (fused), $\beta = (0.9, 0.999)$
Learning rate / schedule	5×10^{-5} , linear, warmup ratio 0.15
Effective batch size	8 (batch size 1, gradient accumulation 8)
Epochs	Up to 120 epochs
Precision	bf16, gradient checkpointing
Checkpoint selection	Lowest validation loss ($\approx$ epoch 16)
Hardware	1 $\times$ RTX 4090 24 GB, ≈ 7 h

required to transport blocks from load zones to unload zones. Blocks must be collected in batches of up to four units and placed such that each block is at least partially within an unload zone (i.e., any overlap/contact with the zone is sufficient). Each block must also be correctly oriented: the upward-facing colored side determines whether the placement is valid. In addition to block transportation, the robot must relocate a movable shelf along the boundary to the designated final side of the field. Successful relocation implicitly requires clearing adjacent zones, although this constraint is not explicitly encoded in the prompts. The episode is considered complete when the robot returns to the finish area.

D. Critic Models and Fine-Tuning

The Qwen2.5-VL-3B critic was adapted via parameter-efficient supervised fine-tuning on 88 execution episodes (65 training and 23 validation), split by recording session. Each sample pairs an annotated field image and its BT execution trace with a target critique following the severity rubric in Section III-C. LoRA adapters were applied to the attention and MLP projections of the 8-bit-quantized base model. The checkpoint with the lowest validation loss, reached near epoch 16, was used in the experiments. Full hyperparameters are reported in Table I.

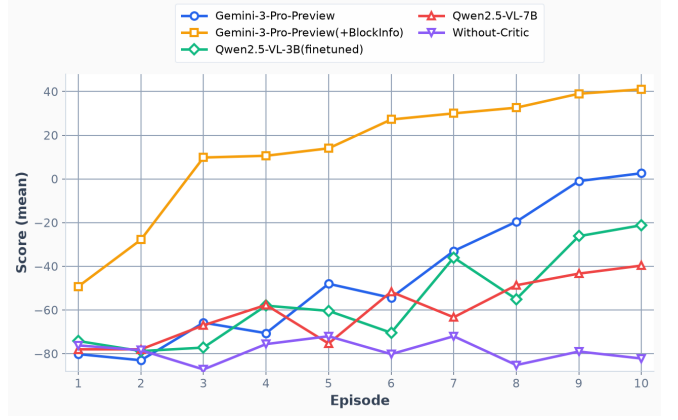
Qwen2.5-VL-7B-Instruct and Gemini-3-Pro-Preview were evaluated without task-specific fine-tuning. All critics remained frozen during episodic policy refinement; only the Behavior Tree was updated.

We evaluate five distinct experimental configurations:

1. Without critic (BT execution trace + human-verified episode score R only)
2. Qwen2.5-VL-7B Instruct
3. Qwen2.5-VL-3B Instruct (fine-tuned)
4. Gemini-3-Pro-Preview (MLLM as critic)
5. Gemini-3-Pro-Preview + block color information

Gemini-3-Pro-Preview was used as a closed-source VLM critic available at the time of data collection. The model was subsequently discontinued on March 9, 2026; therefore, the reported Gemini-based results correspond to the historical preview version used in our experiments and cannot be directly reproduced through the original endpoint.

Each configuration was evaluated on five field layouts with varying block orientations using ten sequential episodes per

Fig. 4. Mean human-verified episode score R across environments.

layout. After each episode, the actor updated the BT using critic feedback and the human-verified score R .

V. EXPERIMENTAL RESULTS

For each VLM, we report results averaged across all environment configurations. We analyze episodic learning behavior using four key metrics: human-verified episode score R , Issue Detection Recall, confidence stability, and human alarm rate. Together, these metrics characterize both the effectiveness of policy refinement and the reliability of visual feedback within VersualRL.

A. Quantitative Learning Results

Human-verified episode score. The human-verified episode score R (Fig. 4) is computed according to a fixed rule-based evaluation function that reflects task completion quality. The scoring scheme, summarized in Table II, is designed to encourage structured logistics behavior and penalize operational errors.

TABLE II

RULES FOR THE HUMAN-VERIFIED EPISODE SCORE R .

Condition	Score
Operating time penalty	-1 point per second
Additional timeout penalty ($t > 150$ s)	-20 points
Block correctly placed	+10 points per block
Full batch of 4 correctly placed blocks	+10 bonus points
Block incorrectly placed	-5 points per block
Block completely outside any zone	-10 points per block
Correct final robot position	+20 points

We first analyze the episodic dynamics of R , which directly reflect task-level performance and policy refinement quality. The fine-tuned Qwen2.5-VL-3B critic demonstrates more consistent improvement in R than the larger Qwen2.5-VL-7B model. Despite its smaller size, the 3B model benefits from task-specific fine-tuning on structured execution feedback, producing more grounded and actionable critiques. In contrast, the untuned 7B model exhibits slower improvement and higher variability, indicating that domain adaptation is more important than model scale in this setting.

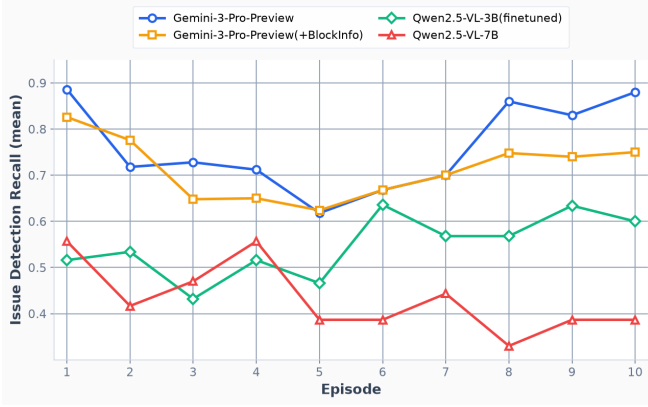


Fig. 5. Mean Issue Detection Recall (IDR) across environments.

Gemini-3-Pro-Preview converges faster and reaches higher final values of R than both Qwen variants, suggesting stronger visual reasoning and feedback quality. Providing symbolic block color information further improves learning stability: Gemini (+BlockInfo) achieves the highest final performance and the most consistent convergence across episodes. This result indicates that structured perceptual inputs reduce ambiguity and support more precise strategy-level corrections.

The no-critic baseline shows substantially slower and less consistent improvement. Without structured visual feedback, the actor relies only on the scalar signal R , which indicates overall task performance but does not reveal the causes of execution failures. Consequently, policy refinement is less targeted and less stable than in the critic-guided configurations.

Issue Detection Recall. Issue Detection Recall (IDR) reflects how reliably the critic detects ground-truth execution issues during task performance (Fig. 5).

We quantify error detection performance using the Issue Detection Recall (IDR):

$$\text{IDR} = \frac{N_{\text{TP}}}{N_{\text{TP}} + N_{\text{FN}}}, \quad (1)$$

where N_{TP} is the number of ground-truth issues detected by the critic, and N_{FN} is the number of ground-truth issues missed by the critic.

Our fine-tuned Qwen2.5-VL-3B model demonstrates higher and more stable recall compared to the larger Qwen2.5-VL-7B model. Despite having fewer parameters, the domain-adapted 3B critic better captures task-specific failure modes, indicating that fine-tuning improves perceptual grounding and structured reasoning about execution errors.

The Qwen2.5-VL-7B model shows lower and more fluctuating recall across episodes, suggesting less consistent alignment between visual observations and task-relevant error categories.

Moving to Gemini-3-Pro-Preview, we observe substantially higher issue detection recall compared to both Qwen

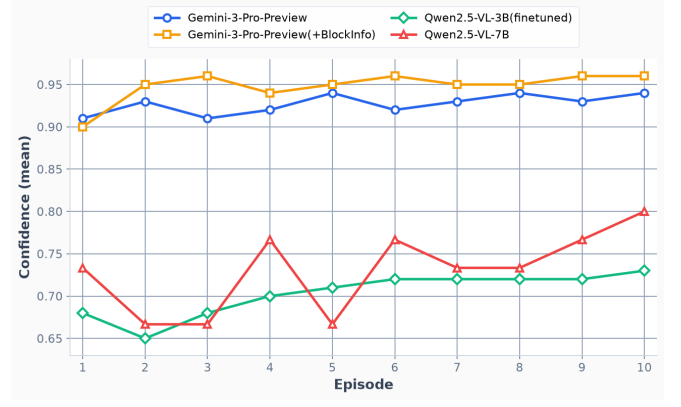


Fig. 6. Mean Confidence across environments.

variants. The model demonstrates strong visual grounding and reliable identification of incorrectly placed or misoriented blocks across most episodes.

When additional symbolic block color information is provided, Gemini (+BlockInfo) does not always achieve higher IDR than the image-only Gemini critic. This is because IDR measures issue detection, not final task performance. BlockInfo mainly helps resolve color/orientation ambiguity and improves actor updates, but remaining errors may involve spatial placement, timing, or manipulation failures. Therefore, the BlockInfo variant can achieve better task scores despite slightly lower issue detection recall in some episodes.

Confidence stability. Confidence reflects how certain the critic is about its assessments during execution (Fig. 6). While recall measures detected issue coverage, confidence stability characterizes how consistently the critic reports its certainty across episodes.

The fine-tuned Qwen2.5-VL-3B model demonstrates more stable confidence levels across episodes compared to the larger Qwen2.5-VL-7B model. The 3B critic maintains moderate but consistent confidence, indicating more stable confidence after task-specific fine-tuning. In contrast, the 7B model exhibits higher variance in confidence, indicating less stable alignment between visual observations and internal reasoning.

The Gemini-3-Pro-Preview critic maintains generally high confidence throughout training, reflecting strong visual grounding capabilities. However, without structured block information, confidence occasionally remains high even in early episodes where planning errors are frequent.

When block color information is provided, Gemini (+BlockInfo) shows both high and more consistent confidence. The additional symbolic input appears to reduce perceptual ambiguity, leading to more stable confidence levels.

Human alarm rate. Human alarm rate measures how often the critic triggers a safety or execution alert requiring human attention (Fig. 7). This signal reflects both error detection reliability and safety awareness during task execution.

Fine-tuned Qwen2.5-VL-3B critic shows a gradual de-

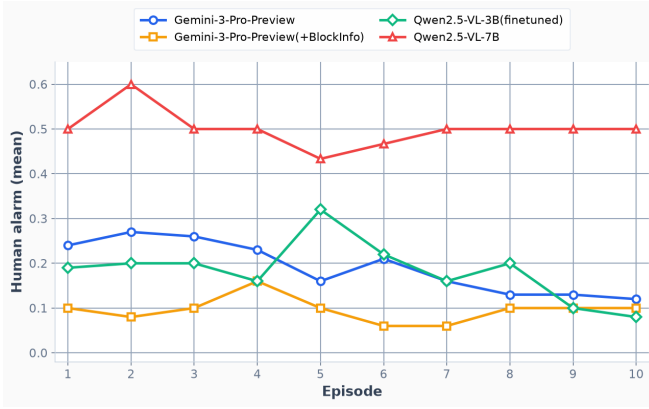


Fig. 7. Mean Human Alarm across environments.

crease in alarm frequency across episodes. As policy quality improves and execution errors become less frequent, the number of triggered alarms correspondingly declines. Since the critic remains frozen, the decreasing alarm rate reflects improvements in the Behavior Tree and a reduction in observable execution failures, rather than adaptation of the critic itself.

In contrast, Qwen2.5-VL-7B produces a higher and more variable alarm rate. The instability suggests less consistent confidence and alarm behavior and less consistent distinction between critical failures and minor execution imperfections.

The Gemini-3-Pro-Preview critic demonstrates a more stable alarm profile, with relatively high alarm rates during early episodes followed by a steady reduction as the policy improves. This behavior is desirable, as it reflects sensitivity to genuine failures during exploration while avoiding excessive intervention later.

The Gemini (+BlockInfo) configuration achieves the most balanced alarm behavior. Early alarms correctly capture orientation and placement errors, while later episodes show a significant reduction in unnecessary alerts. The structured perceptual input improves both detection consistency and safety-related alarm behavior.

Particular attention is given to a representative episode of the Qwen2.5-VL-3B (fine-tuned) critic. In Fig. 8, we show the full robot trajectory during the task, together with the intermediate visual inputs processed by the VLM and the corresponding critic feedback. Based on the structured output of the fine-tuned critic, the system correctly identifies the key failure in the episode and, importantly, attributes it to the appropriate Impact Issue category and related BT node. This precise classification allows the actor to understand not only that an error occurred, but how it affected overall task planning and execution. Such impact-aware attribution is essential for targeted correction rather than global, unfocused replanning.

B. Qualitative BT Evolution Under Perceptual Uncertainty

To complement the aggregate learning curves, Fig. 9 illustrates a representative refinement step where the critic's

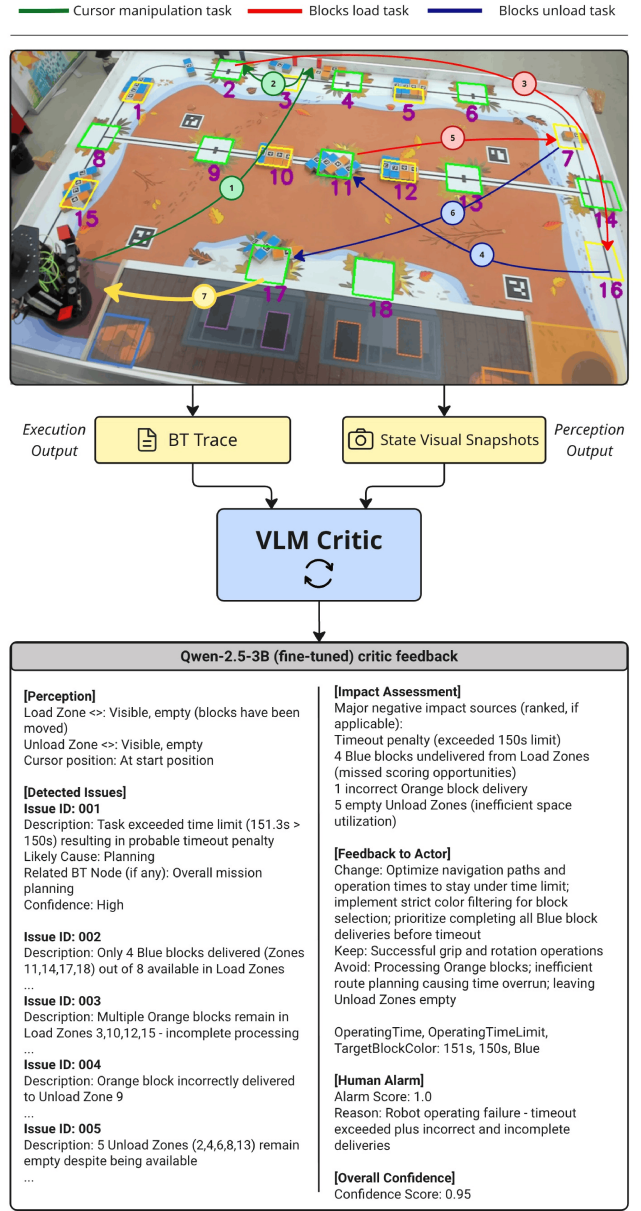


Fig. 8. Episode of the Qwen2.5-VL-3B (fine-tuned) critic.

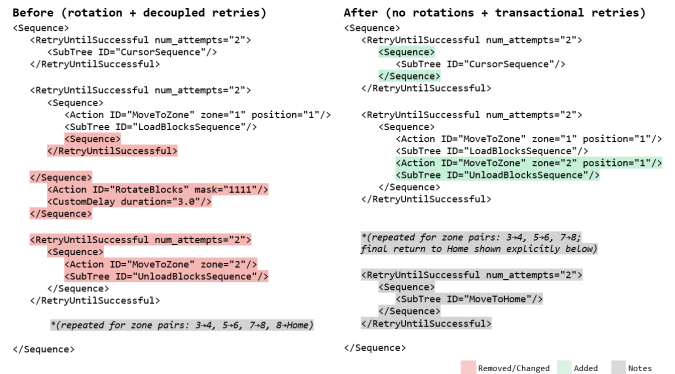


Fig. 9. Qualitative BT evolution under perceptual uncertainty.

perceptual grounding is insufficient for reliable color/orientation reasoning. In this episode, the critic reports empty unload zones and flags a generic execution failure, whereas the BT execution trace and the human-verified episode score R indicate successful transport and placement. The actor therefore treats the critic output as a noisy hypothesis and grounds the update in the BT trace and the human-verified score R , applying a conservative and interpretable BT patch: (i) disabling the blind `RotateBlocks(mask=1111)` routine to avoid time overhead and accidental mis-rotations when color perception is unreliable; (ii) fixing an execution-engine edge case by wrapping `CursorSequence` inside a `Sequence` under `RetryUntilSuccessful`; and (iii) restructuring the policy into isolated transactional `Load--Transport--Unload` subtrees wrapped with `RetryUntilSuccessful`, preventing macro-retries across multiple zone pairs. This behavior also motivates our `BlockInfo` ablation, where reliable symbolic color/orientation estimates enable reintroducing targeted rotations and improve convergence.

VI. DISCUSSION

The experimental results indicate that critic alignment with the task domain plays a more significant role than raw model scale. Although Qwen2.5-VL-3B contains fewer parameters than the 7B variant, the fine-tuned 3B model demonstrates more stable learning dynamics and stronger improvement in R . This suggests that adaptation to structured execution feedback and alignment with Behavior Tree-level error categories are more important than generic visual capacity. The larger untuned model retains broader visual knowledge but lacks tight alignment with the symbolic planning abstraction, resulting in less consistent and less actionable critiques.

Gemini-3-Pro-Preview achieves stronger overall performance, which can be attributed to higher-quality visual grounding and more robust spatial reasoning. The task requires resolving fine-grained spatial relationships, such as identifying block orientation from colored faces, verifying full-batch completion in unload zones, and detecting shelf displacement along field boundaries. Small perceptual errors directly translate into discrete symbolic consequences at the planning level. In particular, accurate recognition of the movable shelf position along the boundary is critical, since even minor misalignment affects the validity of task completion. Gemini’s stronger multimodal reasoning enables more precise and consistent error detection, which in turn enables more effective Behavior Tree refinement. When additional symbolic block color information is provided, perceptual ambiguity is further reduced, shifting the burden from low-level visual inference to structured reasoning. This explains the increased stability and higher final performance of the Gemini (+`BlockInfo`) configuration.

The actor also adapts to the frozen critic’s limitations. As illustrated in Fig. 9, it reduces reliance on perception-sensitive operations when color or zone evidence is unreliable. Thus, adaptation occurs on the actor side: the critic

defines the feedback signal, while the actor adjusts the symbolic strategy accordingly.

Despite these strengths, the system’s performance remains fundamentally constrained by what the critic can observe and attribute within its limited temporal context. The current VersualRL formulation assumes that relevant faults are visible within the critic’s snapshot-based window and can be causally linked to recent BT events. Detecting latent faults with delayed consequences remains challenging, as errors may only manifest several steps later, outside the immediate observation context. Addressing this limitation likely requires longer-horizon temporal reasoning and more explicit trace-aware root-cause attribution.

A second limitation is that VersualRL inherits the critic’s perceptual reliability. When the critic is insufficiently adapted, perceptual mismatches can yield incorrect causal hypotheses. As shown in Fig. 9, the actor can mitigate this by adopting conservative updates that reduce reliance on uncertain perceptual cues, but this may trade off optimality (e.g., disabling rotations that could be beneficial under reliable color observation). Improving critic grounding and incorporating explicit uncertainty handling or structured perceptual inputs (`BlockInfo`) are important directions.

A third limitation is the restricted scope of the experimental evaluation. The framework is validated on a single mobile manipulation platform and a structured warehouse-style task with a fixed library of Behavior Tree nodes. Although the results demonstrate closed-loop adaptation across multiple field configurations, further experiments are needed to assess generalization to unseen layouts, different robot embodiments, and tasks with richer manipulation and interaction dynamics.

Overall, these findings support the central premise of the proposed framework: effective task-level reinforcement in physical robotics does not rely on end-to-end differentiable policy learning, but rather on reliable, structured, and well-aligned verbal feedback grounded in observable execution outcomes. At the same time, they highlight that further improvements in critic grounding, temporal reasoning, and structured feedback design are key directions for advancing Verbal Reinforcement Learning in real-world robotic systems.

Future work will evaluate VersualRL on new robot embodiments, unseen layouts, and richer manipulation tasks; extend the critic with longer video context and trace-aware root-cause attribution; and investigate fully open-source critic configurations to improve reproducibility.

VII. CONCLUSION

We presented VersualRL, a closed-loop verbal reinforcement learning framework with visual execution feedback for task-level policy refinement in mobile robotics. VersualRL addresses the problem of execution uncertainty in stochastic real-world settings by combining a VLM-based critic with an LLM-based actor. The critic generates structured natural-language feedback based on robot observations, allowing the

actor to iteratively refine Behavior Trees without gradient-based optimization.

Key findings from experiments with physical robots include:

1. The human-verified score R alone is insufficient under execution uncertainty, as it cannot distinguish execution (hardware) errors from planning errors. Structured feedback is essential for stable policy improvement.

2. The quality of causal attribution is more important than perceptual accuracy. A critic that correctly diagnoses the causes of failures ensures robust adaptation even with imperfect object detection.

3. Structured perceptual inputs accelerate convergence, demonstrating that the main bottleneck of VersualRL is the perceptual foundation, not symbolic planning.

VersualRL enables interpretable, hardware-based policy adaptation in the real world—without simulators or gradients. Performance depends on the reliability of the critic and the design of the contextual cues, but this framework suggests a promising direction toward transparent robot learning that accounts for the gap between plan and reality.

ACKNOWLEDGEMENTS

Research reported in this publication was financially supported by the RSF grant No. 24-41-02039.

The authors would like to thank Igor Duchinskii, Nikita Kuzmin, Mariya Lezina and Georgii Demianchuk for their valuable assistance with data collection.

REFERENCES

- [1] V. Malathi *et al.*, “Decision-making for path planning of mobile robots under uncertainty: a review of belief-space planning simplifications,” *Robotics*, vol. 14, no. 9, p. 127, 2025.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [3] E. Scheide, G. Best, and G. A. Hollinger, “Behavior tree learning for robotic task planning through Monte Carlo DAG search over a formal grammar,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, May 2021, pp. 4837–4843.
- [4] M. Colledanchise and P. Ögren, “How behavior trees modularize hybrid control systems and generalize sequential behavior compositions, the subsumption architecture, and decision trees,” *IEEE Trans. Robotics*, vol. 33, no. 2, pp. 372–389, 2017.
- [5] R. A. Khan, V. Serpiva, D. Aschalew, A. Fedoseev, and D. Tsetserukou, “Agilepilot: Drl-based drone agent for real-time motion planning in dynamic environments by leveraging object detection,” in *Proc. IEEE Int. Conf. on Unmanned Aircraft System (ICUAS)*, Charlotte, North Carolina, USA, 2025, pp. 185–192.
- [6] L. Ratnabala, R. Peter, A. Fedoseev, and D. Tsetserukou, “Hippo-mat: Decentralized task allocation using graphsage and multi-agent deep reinforcement learning,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, Vienna, Austria, 2026, pp. 1624–1631.
- [7] D. Amodei *et al.*, “Concrete problems in AI safety,” 2016, arXiv:1606.06565.
- [8] B. Ichter *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Proc. Conf. Robot Learning (CoRL)*, 2022, pp. 287–318.
- [9] J. Liang *et al.*, “Code as policies: language model programs for embodied control,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Philadelphia, PA, USA, May 2023, pp. 9493–9500.
- [10] I. Singh *et al.*, “ProgPrompt: program generation for situated robot task planning using large language models,” *Autonomous Robots*, vol. 47, pp. 999–1012, 2023.
- [11] T. Birr, C. Pohl, A. Younes, and T. Asfour, “AutoGPT+P: affordance-based task planning using large language models,” in *Proc. Robotics: Science and Systems (RSS)*, Delft, Netherlands, July 2024, pp. 1–10.
- [12] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: language agents with verbal reinforcement learning,” in *Proc. Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, New Orleans, LA, USA, Dec. 2023, p. Art. no. 377.
- [13] S. Yao, J. Zhao, D. Yu, I. Shafraan, K. R. Narasimhan, and Y. Cao, “ReAct: synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2023.
- [14] A. Madaan *et al.*, “Self-refine: iterative refinement with self-feedback,” in *Proc. 37th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, New Orleans, LA, USA, Dec. 2023, p. Art. no. 2019.
- [15] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: composable 3D value maps for robotic manipulation with language models,” in *Proc. Annu. Conf. Robot Learning (CoRL)*, 2023, pp. 540–562.
- [16] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, “Task and motion planning with large language models for object rearrangement,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Detroit, MI, USA, Oct. 2023, pp. 2086–2092.
- [17] W. Huang *et al.*, “Inner monologue: embodied reasoning through planning with language models,” in *Proc. Conf. Robot Learning (CoRL)*, Auckland, New Zealand, Dec. 2022, pp. 1769–1782.
- [18] A. Lykov *et al.*, “LLM-MARS: large language model for behavior tree generation and NLP-enhanced dialogue in multi-agent robot systems,” 2023, arXiv:2312.09348.
- [19] S. Liu, J. Zhang, R. X. Gao, X. V. Wang, and L. Wang, “Vision-language model-driven scene understanding and robotic object manipulation,” in *Proc. IEEE Int. Conf. Automation Science and Engineering (CASE)*, Aug. 2024, pp. 21–26.
- [20] J. Wang *et al.*, “Large language models for robotics: opportunities, challenges, and perspectives,” *J. Autom. Intell.*, vol. 4, pp. 52–64, 2025.
- [21] G. Wang *et al.*, “Voyager: an open-ended embodied agent with large language models,” *Trans. Mach. Learn. Res.*, vol. 2024, 2024.
- [22] R. A. Izzo, G. Bardaro, and M. Matteucci, “BTGenBot: behavior tree generation for robotic tasks with lightweight LLMs,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Abu Dhabi, UAE, Oct. 2024, pp. 9684–9690.
- [23] A. Lykov and D. Tsetserukou, “LLM-BRAIN: AI-driven fast generation of robot behavior tree based on large language model,” in *Proc. IEEE Int. Conf. Foundation and Large Language Models (FLLM)*, Dubai, UAE, Nov. 2024, pp. 392–397.
- [24] Y. Wang *et al.*, “RL-VLM-F: reinforcement learning from vision language foundation model feedback,” in *Proc. Int. Conf. Machine Learning (ICML)*, Vienna, Austria, July 2024, p. Art. no. 2112.
- [25] J. Rocamonde, V. Montesinos, E. Nava, E. Perez, and D. Lindner, “Vision-language models are zero-shot reward models for reinforcement learning,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2024.
- [26] A. Singh, *et al.*, “VARP: reinforcement learning from vision-language model feedback with agent regularized preferences,” 2025, arXiv:2503.13817.
- [27] X. Dang and S. Edelkamp, “CLIP-Motion: learning reward functions for robotic actions using consecutive observations,” 2023, arXiv:2311.03485.
- [28] Y. Zeng, Y. Mu, and L. Shao, “Learning reward for robot skills using large language models via self-alignment,” in *Proc. Int. Conf. Machine Learning (ICML)*, Vienna, Austria, 2024, p. Art. no. 2408.
- [29] Z. Guo, A. Lykov, Z. Yagudin, M. Konenkov, and D. Tsetserukou, “VLM-Auto: VLM-based autonomous driving assistant with human-like behavior and understanding for complex road scenes,” in *Proc. IEEE Int. Conf. Foundation and Large Language Models (FLLM)*, Dubai, UAE, Nov. 2024, pp. 501–507.
- [30] O. Y. Lee, A. Xie, K. Fang, K. Pertsch, and C. Finn, “Affordance-guided reinforcement learning via visual prompting,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Hangzhou, China, 2025, pp. 2352–2359.
- [31] L. Guan, Y. Zhou, D. Liu, Y. Zha, H. B. Amor, and S. Kambhampati, “Task success is not enough: investigating the use of video-language models as behavior critics for catching undesirable agent behaviors,” in *Proc. Conf. Language Modeling (COLM)*, Philadelphia, PA, USA, Oct. 2024, pp. 1–15.
- [32] R. A. Khan, Y. Yaqoot, A. A. Habel, M. A. Mustafa, and D. Tsetserukou, “AgenticRL: self-refining agentic reinforcement learning for vision-conditioned UAV navigation,” 2026, arXiv:2606.03963.